

Enunciados Expandidos

Seção 5 — Estudos de Caso e Exercícios Resolvidos, mais um desafio extra (5.4) | Programação Orientada a Objetos em Java | Prof. Dani Juliano Czelusniak

Este material reescreve os três enunciados originais (5.1, 5.2 e 5.3) de forma mais detalhada, quebrando os requisitos em partes menores e trazendo perguntas-guia para ajudar no raciocínio. Inclui também um desafio extra (5.4), que combina o exercício da ContaBancaria com o Sistema de Vendas, e um apêndice explicando try/catch. Nenhum código-solução é apresentado — o objetivo é ajudar a visualizar o que cada exercício pede antes de começar a programar.

5.1 — Sistema de Vendas (Composição Simples)

Enunciado original

Implemente um sistema composto por **Produto** e **ItemPedido**. O item de pedido deve calcular seu subtotal invocando informações do produto. Utilize validação para barrar quantidades negativas. Crie a classe de teste principal para rodar a simulação e instanciar os objetos.

Expandindo

Você precisa modelar duas classes que se relacionam por **composição** (um objeto guardando outro como atributo, como no exemplo Segmento_de_Reta que guarda dois Ponto).

- **Produto:** precisa guardar um código e um preço. Pense: esses atributos devem ser public ou private? Se forem private, como o ItemPedido vai conseguir ler o preço lá de dentro?
- **ItemPedido:** não guarda o preço diretamente — ele guarda uma referência a um objeto Produto, mais a quantidade comprada. O subtotal não é um atributo fixo, é algo calculado na hora por um método, usando a quantidade e o preço do produto que ele carrega.
- **Validação:** quantidade não pode ser negativa (nem zero, dependendo de como você interpretar). Onde essa checagem deve acontecer — no construtor, num método separado, ou nos dois?
- **Classe de teste (main):** precisa criar um Produto, criar um ItemPedido referenciando esse produto, mostrar o subtotal, e também testar o caso de erro (quantidade inválida).

Pergunta pra pensar: por que faz sentido o ItemPedido não duplicar o preço do produto dentro de si, e sim ir buscar essa informação toda vez que precisar calcular o subtotal?

5.2 — Encapsulamento Avançado em Veículos

Enunciado original

Modele uma **Motocicleta** para turismo onde o estado do motor (ligado/desligado) e a velocidade precisem ser rigorosamente controlados. O piloto só pode alterar a velocidade se o motor estiver em funcionamento, garantindo a coesão das regras de negócio (encapsulamento total).

Expandindo

Aqui o foco não é composição, é **encapsulamento** puro — a classe precisa proteger seu próprio estado interno contra alterações inválidas, e quem usa a classe só pode mexer nela através de

métodos controlados, nunca acessando o atributo diretamente.

- **Atributos:** modelo, se o motor está ligado (que tipo de dado em Java só tem duas possibilidades?), velocidade atual, e um limite de velocidade.
- **Métodos de comportamento:** um para ligar o motor, e um para acelerar.
- **Regra central:** acelerar só pode funcionar se o motor estiver ligado. O método de acelerar precisa checar isso logo no início. O que ele deve fazer se o motor estiver desligado — lançar uma exceção, ou apenas avisar e não fazer nada?
- **Limite de segurança:** o que acontece se o piloto tentar acelerar além do limite da moto? A velocidade deve ultrapassar ou travar num teto?

Pergunta pra pensar: por que "encapsulamento total" aqui significa que nem o construtor pode deixar a moto nascer com o motor ligado por acidente?

5.3 — Interação Complexa no Ambiente Acadêmico

Enunciado original

*Simule a interação entre um **Professor** e um **Estudante**. O objeto da classe Professor deve realizar uma operação que modifica ativamente o estado do objeto Estudante (atribuir uma nota), demonstrando a mensageria direta entre diferentes entidades do sistema.*

Expandindo

Esse exercício é sobre **mensageria entre objetos** — um objeto chamando um método de outro objeto, e não mexendo direto no atributo dele.

- **Estudante:** tem nome e uma nota final. A nota deve ter alguma validação (pense num intervalo razoável, tipo 0 a 10).
- **Professor:** tem nome, e um método que "avalia" um estudante. A parte chave: esse método recebe um objeto Estudante como parâmetro (igual o Segmento_de_Reta recebia dois Ponto no construtor).
- Dentro desse método, o professor não faz `estudante.nota = valor` diretamente (isso quebraria o encapsulamento) — ele chama um método do próprio Estudante para que o estudante registre a própria nota.
- Isso é o mesmo padrão de "A envia mensagem para B" descrito no material com o exemplo do Jogo da Velha (Jogador chama `marcar(x,y)` no Tabuleiro, ao invés de mexer direto nas coordenadas do tabuleiro).

Pergunta pra pensar: por que é mais correto o Professor pedir para o Estudante "se auto-atualizar" (chamando um método dele) do que o Professor alterar a nota do estudante diretamente?

5.4 — Desafio Extra: Compra com Verificação de Saldo

Origem do desafio

*Este exercício não está no material original do professor — foi proposto como extensão, combinando dois exercícios já feitos: a **ContaBancaria** (Fundamentos de Classes e Objetos) e o **Sistema de Vendas** (5.1, Produto + ItemPedido).*

Enunciado

Integre a classe **ContaBancaria** com as classes **Produto** e **ItemPedido** já construídas. Antes de finalizar a compra de um **ItemPedido**, o sistema deve verificar se o saldo da conta é suficiente para cobrir o subtotal do pedido. Se não for suficiente, a compra deve ser recusada de forma controlada (sem derrubar o programa), exibindo uma mensagem de erro amigável.

Expandindo

- **Quem decide se a compra é aprovada?** Pense em qual dos três objetos (**ContaBancaria**, **Produto**, **ItemPedido**) faz mais sentido tomar essa decisão — compare com o padrão Professor/Estudante do exercício 5.3, em que quem inicia a ação manda uma mensagem para o outro objeto.
- **Reaproveite o que já existe.** A validação de "valor maior que o saldo" já foi resolvida dentro do método de saque da **ContaBancaria**. Não é necessário reescrever essa lógica em outro lugar — pense em como reutilizá-la.
- **Ordem das operações:** primeiro é preciso saber quanto custa o pedido (isso já existe em **ItemPedido**) antes de tentar debitar da conta.
- **Tratamento de erro:** quando o saldo não for suficiente, uma exceção é lançada. O programa não pode travar por causa disso — é aqui que entra o try/catch (veja o apêndice a seguir).

Pergunta pra pensar: por que reaproveitar o método de saque já existente é melhor do que escrever a verificação "subtotal maior que saldo" de novo em outro método?

Apêndice — Entendendo o try/catch

Quando um método lança uma exceção (por exemplo, o `throw new IllegalArgumentException(...)` usado em **ContaBancaria**) e ninguém trata essa exceção, o programa é interrompido imediatamente e o Java imprime um stack trace no console. O bloco **try/catch** serve para interceptar essa exceção antes que ela derrube o programa, permitindo decidir o que fazer a seguir.

Estrutura básica

```
try {  
    // código que PODE lançar uma exceção  
} catch (TipoDaExcecao nomeDaVariavel) {  
    // código que roda SE (e só se) a exceção acontecer  
}
```

Como funciona

- O Java executa o bloco try linha por linha, normalmente.
- Se uma instrução dentro do try lançar uma exceção do tipo esperado pelo catch, a execução pula imediatamente para dentro do catch — nenhuma linha restante do try é executada.
- Se nenhuma exceção acontecer, o catch inteiro é ignorado e o programa continua normalmente depois dele.
- O método `e.getMessage()` devolve o texto passado no `throw new IllegalArgumentException(...)` que originou a exceção.

Exemplo (já presente no material do professor, seção 1.2)

```
try {
    ContaBancaria cb = new ContaBancaria(100.0);
    System.out.println("Saldo inicial: R$ " + cb.getSaldo());

    cb.depositar(50.0);
    System.out.println("Saldo apos deposito: R$ " + cb.getSaldo());

    // Forcando erro com deposito negativo para acionar validacao
    cb.depositar(-20.0);

} catch (IllegalArgumentException e) {
    System.out.println("Erro na operacao: " + e.getMessage());
}
```

Pergunta pra pensar: no seu próprio código, qual chamada de método deveria ficar dentro do try, e o que faz sentido mostrar dentro do catch quando a compra de um ItemPedido falha por falta de saldo?

Documento de apoio gerado a partir do material "02_Fundamentos_POO.pdf" e da conversa de estudo — resumos, desafios extras e perguntas-guia elaborados para orientar o raciocínio, sem apresentar o código-solução.